

Team - EPOCH

**Khushi Saxena,
Archit Gautam**

ADOBE INDIA HACKATHON

Connecting the dots challenge

OVERVIEW:

What Is This Project About?

The Adobe India Hackathon 2025 revolves around the "Connecting the Dots Challenge" - a groundbreaking effort to revolutionize the way we engage with PDF files. Rather than viewing PDFs as immobile, dead documents, this project sees them as smart, interactive friends that can:

- Talk to users and tell stories meaningfully.
- Link concepts across whole document collections.
- Surface insights automatically from complex content.
- Respond intelligently like a trusted research assistant.

The Vision:

The Big Question: *"What if every time you opened a PDF, it didn't just sit there—it spoke to you, connected ideas, and narrated meaning across your entire library?"*

The Mission: Building the future of document interaction by developing PDF experiences that are magical through:

- Smart content comprehension
- Cross-document relationship mapping
- Simple user interfaces
- Context-aware suggestions

CHALLENGE 1 A: PDF Processing Solution

Overview: This project outlines an automated approach to get structured outlines from PDF files, for Adobe India Hackathon 2025 (Challenge 1A). The approach intelligently extracts document titles and hierarchical headings and provides the structure as standardized JSON files.

Designed to execute within Docker containers, the system is created to perform under stringent performance limits, such as limited cross-platform compatibility, CPU, and memory. It aims to provide strong functionality both for simple and complex PDFs.

Key Features:

- Automated Pipeline: Scans and processes all PDFs found in the /app/input folder.
- Structured JSON Output: Produces a filename.json for every filename.pdf, maintaining heading levels and page mapping.
- Hierarchical Detection: Correctly detects titles, H1/H2/H3 headings, and document structure.
- Containerized Deployment: Completely Dockerized to maintain consistency and environment segregation.
- Resource Efficient: Optimized for sub-10 second processing within 16GB RAM and 8-core CPU constraints.
- Multilingual Support: Enhanced Japanese language processing with Unicode normalization.
- Intelligent Text Block Extraction: Extracts formatted text with font information and positioning.
- Smart Heading Detection: Identifies document structure with language-specific patterns.
- Batch Processing: Concurrent processing of multiple PDFs using ThreadPoolExecutor.
- Docker Integration: Container-ready with predefined input/output directories.

Section-by-Section Analysis:

1. Import Statements and Dependencies :

```
import fitz # PyMuPDF
import os
import json
import logging
import re
import time
import unicodedata
from typing import List, Dict, Optional, Tuple
from concurrent.futures import ThreadPoolExecutor, as_completed
```

Purpose: Establishes the foundation for the application

- **PyMuPDF (fitz):** Core PDF processing library for text extraction
- **unicodedata:** Handles Unicode text normalization for proper Japanese character processing
- **concurrent.futures:** Enables parallel processing for batch operations.
- **typing:** Provides type hints for better code documentation and IDE support.

2. Logging Configuration :

```
# -----
# Logging Setup
# -----
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)
logger = logging.getLogger(__name__)
```

Purpose: Sets up comprehensive logging system

- Configures INFO-level logging with timestamp formatting
- Provides tracking for processing status, errors, and performance metrics
- Essential for debugging and monitoring batch operations

3. Language Detection Utilities : -

3.1 Language Detection Function :

```
def detect_language(text: str) -> str:
    """
    Simple language detection based on character sets.
    Returns 'ja' for Japanese, 'en' for English.
    """
    if not text:
        return 'en'

    # Count Japanese characters
    japanese_chars = 0
    total_chars = 0

    for char in text:
        if char.isspace():
            continue
        total_chars += 1

        # Check for Hiragana, Katakana, and Kanji
        if (0x3040 <= ord(char) <= 0x309F or # Hiragana
            0x30A0 <= ord(char) <= 0x30FF or # Katakana
            0x4E00 <= ord(char) <= 0x9FFF or # Kanji (CJK Unified Ideographs)
            0x3400 <= ord(char) <= 0x4DBF): # Kanji Extension A
            japanese_chars += 1

    if total_chars == 0:
        return 'en'

    japanese_ratio = japanese_chars / total_chars
    return 'ja' if japanese_ratio > 0.3 else 'en'
```

Purpose: Automatically identifies whether text is Japanese or English.

- **Algorithm:** Analyzes character Unicode ranges to count Japanese characters.
- **Character Sets Detected:**
 - Hiragana (0x3040-0x309F)
 - Katakana (0x30A0-0x30FF)
 - Kanji/CJK Unified Ideographs (0x4E00-0x9FFF)
 - Kanji Extension A (0x3400-0x4DBF)
- **Threshold:** Returns 'ja' if >30% of characters are Japanese, otherwise 'en'.

3.2 Japanese Text Normalization :

```
def normalize_japanese_text(text: str) -> str:
    """
    Normalize Japanese text for better processing.
    """
    # Normalize Unicode (NFC normalization)
    text = unicodedata.normalize('NFC', text)

    # Convert full-width numbers and letters to half-width
    text = text.translate(str.maketrans(
        '0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz',
        '0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz'
    ))

    # Convert full-width spaces to half-width
    text = text.replace('　', ' ')

    return text.strip()
```

Purpose: Standardizes Japanese text for consistent processing

- **Unicode Normalization:** Applies NFC (Canonical Decomposition followed by Canonical Composition).
- **Character Width Conversion:** Converts full-width numbers/letters to half-width equivalents.
- **Space Standardization:** Replaces full-width spaces () with standard spaces.
- **Impact:** Ensures consistent text representation across different input sources.

4. Text Block Extraction :

```
def extract_text_blocks(pdf_path: str) -> List[Dict]:
    try:
        doc = fitz.open(pdf_path)
        text_blocks = []

        for page_num, page in enumerate(doc, start=1):
            page_width = page.rect.width
            page_dict = page.get_text("dict")
            blocks = page_dict["blocks"]

            for block in blocks:
                if "lines" not in block:
                    continue

                for line in block["lines"]:
                    for span in line["spans"]:
                        text = span["text"].strip()
                        if not text:
                            continue
```

```
        # Normalize Japanese text
        normalized_text = normalize_japanese_text(text)
        if not normalized_text:
            continue

        font_flags = span.get("flags", 0)
        is_bold = bool(font_flags & (1 << 4)) or 'Bold' in span["font"]
        is_italic = bool(font_flags & (1 << 1)) or 'Italic' in span["font"]

        # Detect language
        language = detect_language(normalized_text)

        text_blocks.append({
            "text": normalized_text,
            "original_text": text,
            "font_size": round(span["size"], 2),
            "font": span["font"],
            "bold": is_bold,
            "italic": is_italic,
            "x": round(span["bbox"][0], 2),
            "y": round(span["bbox"][1], 2),
            "page": page_num,
            "page_width": page_width,
            "language": language,
        })

    doc.close()
    return text_blocks
except Exception as e:
    logger.error(f"✖ Error processing {pdf_path}: {e}")
    return []
```

Purpose: Core extraction function that processes PDF pages and extracts formatted text blocks.

Process Flow:

- **PDF Opening:** Uses PyMuPDF to open and iterate through pages.
- **Dictionary Extraction:** Retrieves structured text data with formatting information.
- **Block Processing:** Processes each text block, line, and span
- **Font Analysis:** Extracts font properties including:
 - Font size and family
 - Bold/italic formatting (using bit flags)
 - Positioning coordinates (bounding box)
- **Text Enhancement:** Applies Japanese normalization and language detection.
- **Data Structure:** Returns comprehensive text block information.

5. Text Block Merging System :

```
# -----
def merge_blocks(blocks, y_tolerance=3, x_gap_threshold=10):
    if len(blocks) == 0:
        return []

    blocks = sorted(blocks, key=lambda b: (round(b['y']), b['x']))
    merged = []
    current = blocks[0].copy()

    for blk in blocks[1:]:
        same_line = abs(blk['y'] - current['y']) <= y_tolerance
        same_style = blk['font_size'] == current['font_size'] and blk['bold'] == current['bold']
        same_font = blk['font'] == current['font']
        same_language = blk['language'] == current['language']

        # Adjust gap calculation for Japanese (characters can be closer)
        if current['language'] == 'ja':
            current_text_width = len(current['text']) * current['font_size'] * 0.8 # Japanese chars are wider
            gap_threshold = x_gap_threshold * 1.5 # Allow larger gaps for Japanese
        else:
            current_text_width = len(current['text']) * current['font_size'] * 0.6
            gap_threshold = x_gap_threshold

        gap = blk['x'] - (current['x'] + current_text_width)
        adjacent_x = 0 <= gap <= gap_threshold

        if same_line and same_style and same_font and same_language and adjacent_x:
            # Add appropriate spacing for Japanese
            if current['language'] == 'ja':
                # Japanese doesn't always need spaces between merged text
                current['text'] += blk['text']
            else:
                current['text'] += " " + blk['text']
        else:
            merged.append(current)
            current = blk.copy()

    merged.append(current)
    return merged
```

Purpose: Intelligently combines fragmented text blocks into coherent units

Merging Logic:

- Spatial Analysis: Groups blocks on the same line (Y-coordinate tolerance).
- Style Consistency: Merges blocks with matching font size, weight, and family.
- Language Awareness: Considers language-specific spacing requirements.
- Gap Calculation:
 - Japanese: Allows larger gaps (1.5x threshold) due to character width.
 - English: Uses standard character width calculations.
- Spacing Rules:
 - Japanese text: No spaces added between merged segments.
 - English text: Adds spaces between merged segments.

Benefits: Reconstructs fragmented text that may be split across multiple PDF elements.

6. Title Detection System :

```
def detect_title(text_blocks: List[Dict]) -> Tuple[Optional[str], Optional[float], Optional[str]]:
    first_page_blocks = [b for b in text_blocks if b['page'] == 1 and b['font_size'] >= 10]
    if not first_page_blocks:
        return None, None, None

    max_font_size = max(b['font_size'] for b in first_page_blocks)
    candidate_blocks = [b for b in first_page_blocks if b['font_size'] == max_font_size and b['y'] <= 200]

    if not candidate_blocks:
        return None, None, None

    # Find centered candidates with language-aware centering
    centered_candidates = []
    for b in candidate_blocks:
        if b['language'] == 'ja':
            # Japanese text centering calculation
            approx_center_x = b['x'] + (len(b['text']) * b['font_size'] * 0.4)
        else:
            approx_center_x = b['x'] + (len(b['text']) * b['font_size'] * 0.3)

        page_center = b['page_width'] / 2
        if abs(approx_center_x - page_center) < 100:
            centered_candidates.append(b)

    if not centered_candidates:
        centered_candidates = candidate_blocks

    merged = merge_blocks(centered_candidates)
    if not merged:
        return None, None, None

    merged.sort(key=lambda b: b['y'])
    title_text = merged[0]['text'].strip()
    title_language = merged[0]['language']
    return title_text, max_font_size, title_language
```

Purpose: Automatically identifies document titles using multiple heuristics.

Detection Strategy:

1. Page Filtering: Focuses on first page content.
2. Font Size Analysis: Identifies largest font size as potential title indicator.
3. Position Analysis: Looks for text in upper portion of page ($Y < 200$).
4. Centering Detection:
 - Language-specific center calculations
 - Japanese: Uses 0.4x character width multiplier
 - English: Uses 0.3x character width multiplier
5. Fallback Logic: Uses largest font candidates if no centered text found.

Returns: Title text, font size, and detected language.

7. Advanced Heading Detection System:

7.1 Heading Level Detection:

```
def heading_level_from_text(text: str, language: str, default_level: str = "H4") -> str:
    """
    Detect heading level from text content with language-specific patterns.
    """
    cleaned_text = text.strip().lower()

    if language == 'ja':
        # Japanese patterns
        # Chapter patterns: 第1章, 第一章, 1章, etc.
        if re.search(r'第[一二三四五六七八九十\d]+章', text):
            return "H1"
        if re.search(r'[一二三四五六七八九十\d]+章', text):
            return "H1"

        # Section patterns: 第1節, 1節, etc.
        if re.search(r'第[一二三四五六七八九十\d]+節', text):
            return "H2"
        if re.search(r'[一二三四五六七八九十\d]+節', text):
            return "H2"

        # Japanese unit patterns
        if re.search(r'[一二三四五六七八九十\d]+单元', text):
            return "H2"
        if re.search(r'[一二三四五六七八九十\d]+单元', text):
            return "H2"

        # Common Japanese headings
        if any(word in text for word in ['目的', '目標', 'ねらい', '学習目標']):
            return "H2"
        if any(word in text for word in ['概要', 'まとめ', '要約', 'おわりに']):
            return "H2"
        if any(word in text for word in ['はじめに', '序論', '導入']):
            return "H2"

        # Numbered patterns: 1. 2. 3. or 1. 2. 3.
        if re.match(r'^[1-9一二三四五六七八九十][1 2 3 4 5 6 7 8 9 0][\.\. ]', text):
            return "H3"
```



```

else:
    # English patterns (original logic)
    if re.match(r"^(r\d{2}[a-z]+\d{4})\s+.+$", text, re.IGNORECASE):
        return "H1"
    if "objectives" in cleaned_text:
        return "H2"
    if "outcomes" in cleaned_text:
        return "H2"
    if re.match(r"^unit[\s-]*[iivxcl\d]+$", cleaned_text, re.IGNORECASE):
        return "H3"
    if re.match(r"^(chapter|module)[\s-]*[iivxcl\d]+$", cleaned_text, re.IGNORECASE):
        return "H3"

return default_level

```

Purpose: Analyzes text content to determine appropriate heading hierarchy level

Japanese Pattern Recognition:

- **Chapter Patterns:** 第1章, 第一章, 1章 → H1
- **Section Patterns:** 第1節, 1節 → H2
- **Unit Patterns:** 第1单元, 1单元 → H2
- **Semantic Keywords:** 目的, 目標, ねらい, 概要, まとめ → H2
- **Numbered Lists:** 1. 2. 3. or 1. 2. 3. → H3

English Pattern Recognition:

- **Reference Codes:** (r02abc2024) format → H1
- **Academic Keywords:** "objectives", "outcomes" → H2
- **Structural Units:** "Unit I", "Chapter 1" → H3.

7.2 Heading Likelihood Assessment :

```
def is_likely_heading(text: str, language: str) -> bool:
    """
    Determine if text is likely a heading with language-specific rules.
    """
    if not text or len(text.strip()) == 0:
        return False

    if language == 'ja':
        # Japanese heading criteria
        # Length check (Japanese headings can be longer due to character density)
        if len(text) > 150:
            return False

        # Japanese headings rarely end with these characters
        if text.endswith(('。', '、', '?', '？')):
            return False

        # Check for typical Japanese heading patterns
        has_chapter_pattern = bool(re.search(r'第?[一二三四五六七八九十\d]+[章節单元]', text))
        has_numbered_pattern = bool(re.match(r'^[1-9一二三四五六七八九十]+[1 2 3 4 5 6 7 8 9 0][\.\. ]', text))
        has_heading_words = any(word in text for word in [
            '目的', '目標', 'ねらい', '概要', 'まとめ', '要約',
            'はじめに', '序論', '導入', 'おわりに', '結論'
        ])

        # More lenient for Japanese - check character density and patterns
        if has_chapter_pattern or has_numbered_pattern or has_heading_words:
            return True

        # For other Japanese text, be more restrictive about length
        if len(text) > 50:
            return False

    else:
        # English criteria (original logic)
        if len(text) > 100 or text.count(" ") > 15:
            return False
        if text.endswith(".") or text.endswith("?"):
            return False
        if text[0].islower():
            return False

    return True
```

Purpose: Filters potential headings using language-specific criteria

Japanese Criteria:

- Length limit: 150 characters (accommodates Japanese character density)
- Exclusion patterns: Avoids text ending with 。 、 ？
- Pattern matching: Validates chapter/section/unit patterns
- Keyword recognition: Identifies common heading vocabulary

English Criteria:

- Length limit: 100 characters or 15 words
- Format validation: Excludes sentences ending with periods/questions

7.3 Main Heading Detection :

```
def detect_headings(text_blocks: List[Dict], title_font_size: Optional[float]) -> List[Dict]:
    excluded_size = {title_font_size} if title_font_size else set()
    min_heading_size = 12

    heading_blocks = [
        b for b in text_blocks
        if b['font_size'] not in excluded_size and
           b['font_size'] >= min_heading_size and
           b['bold'] and
           b['y'] < 700
    ]

    # Group by both font size and language
    grouped_by_font_lang = {}
    for b in heading_blocks:
        key = (b['font_size'], b['language'])
        grouped_by_font_lang.setdefault(key, []).append(b)

    # Sort sizes within each language group
    sizes_sorted = sorted(set(fs for fs, lang in grouped_by_font_lang.keys()), reverse=True)
    level_names = ["H1", "H2", "H3", "H4"]

    final_headings = []

    for (font_size, lang) in grouped_by_font_lang:
        size_rank = sizes_sorted.index(font_size)
        if size_rank < len(level_names):
            default_level = level_names[size_rank]
        else:
            default_level = "H4"

        group = grouped_by_font_lang[(font_size, lang)]
        merged = merge_blocks(group)

        for b in merged:
            if is_likely_heading(b['text'], b['language']):
                detected_level = heading_level_from_text(b['text'], b['language'], default_level)
                final_headings.append({
                    "level": detected_level,
                    "text": b['text'].strip(),
                    "page": b['page'],
                    "y": b['y'],
                    "language": b['language']
                })
```

Purpose: Orchestrates the complete heading detection process

Process:

1. Filtering: Selects bold text blocks above minimum size threshold.
2. Grouping: Groups candidates by font size and language.
3. Ranking: Assigns hierarchy levels based on font size ranking.
4. Validation: Applies likelihood filtering and content analysis.
5. Sorting: Orders headings by page and vertical position.

8. PDF Processing Pipeline:

```
def process_pdf(pdf_path: str) -> Dict:
    logger.info(f"📄 Processing: {os.path.basename(pdf_path)}")
    start_time = time.time()

    blocks = extract_text_blocks(pdf_path)
    if not blocks:
        return {"title": "", "title_language": "en", "outline": [], "document_language": "en"}

    title, title_font_size, title_language = detect_title(blocks)
    headings = detect_headings(blocks, title_font_size)

    # Determine dominant document language
    languages = [b['language'] for b in blocks]
    document_language = max(set(languages), key=languages.count) if languages else 'en'

    elapsed = time.time() - start_time
    logger.info(f"🕒 Processed {os.path.basename(pdf_path)} in {elapsed:.2f} seconds")

    return {
        "title": title or "",
        "title_language": title_language or document_language,
        "outline": headings,
        "document_language": document_language
    }
```

Purpose: Main processing function that coordinates all extraction operations.

Workflow:

1. Text Extraction: Extracts all text blocks from PDF.
2. Title Detection: Identifies document title and font characteristics.
3. Heading Analysis: Detects and categorizes document structure.
4. Language Analysis: Determines dominant document language.
5. Performance Tracking: Logs processing time and status.

Output Structure:

```
{
    "title": title or "",
    "title_language": title_language or document_language,
    "outline": headings,
    "document_language": document_language
}
```

9. Batch Processing System :

```
def process_pdfs(input_dir: str, output_dir: str, max_workers: int = 4) -> None:
    start_batch = time.time()
    os.makedirs(output_dir, exist_ok=True)

    # Ensure directories exist
    if not os.path.exists(input_dir):
        logger.error(f"❌ Input directory does not exist: {input_dir}")
        return

    pdfs = [f for f in os.listdir(input_dir) if f.lower().endswith(".pdf")]
    if not pdfs:
        logger.warning(f"⚠️ No PDFs found in {input_dir}")
        return

    logger.info(f"🔍 Found {len(pdfs)} PDF(s) to process")

    def worker(file: str) -> Tuple[str, bool]:
        try:
            input_path = os.path.join(input_dir, file)
            output_path = os.path.join(output_dir, os.path.splitext(file)[0] + ".json")

            result = process_pdf(input_path)

            with open(output_path, "w", encoding="utf-8") as f:
                json.dump(result, f, indent=4, ensure_ascii=False)

            logger.info(f"✅ Output: {os.path.basename(output_path)}")
            return (file, True)
        except Exception as e:
            logger.error(f"❌ Failed on {file}: {e}")
            return (file, False)

    successful = 0
    failed = 0
```

```
successful = 0
failed = 0

with ThreadPoolExecutor(max_workers=max_workers) as executor:
    futures = [executor.submit(worker, file) for file in pdfs]
    for future in as_completed(futures):
        _, success = future.result()
        if success:
            successful += 1
        else:
            failed += 1

total_batch_time = time.time() - start_batch
logger.info(f"🏁 Completed processing: {successful} successful, {failed} failed")
logger.info(f"🕒 Total batch time: {total_batch_time:.2f} seconds")
```

Purpose: Handles concurrent processing of multiple PDF files

Features:

- **Directory Management:** Validates input/output directories
- **File Discovery:** Identifies all PDF files in input directory
- **Parallel Processing:** Uses ThreadPoolExecutor for concurrent operations
- **Error Handling:** Continues processing despite individual file failures
- **Progress Tracking:** Logs success/failure statistics
- **Output Generation:** Creates JSON files with structured results

Performance: Configurable worker threads (default: 4) for optimal resource utilization.

10. Docker Integration :

```
def main():
    start = time.time()

    # Docker paths
    input_dir = "/app/input"
    output_dir = "/app/output"

    # Ensure directories exist
    os.makedirs(input_dir, exist_ok=True)
    os.makedirs(output_dir, exist_ok=True)

    logger.info("🚀 Starting PDF processing in Docker container")
    logger.info(f"📁 Input directory: {input_dir}")
    logger.info(f"📁 Output directory: {output_dir}")

    max_threads = 4
    process_pdfs(input_dir, output_dir, max_workers=max_threads)

    end = time.time()
    total_time = end - start
    logger.info(f"✅ Total execution time: {total_time:.2f} seconds")
    logger.info(f"🎉 PDF extraction complete.")

if __name__ == "__main__":
    main()
```

Purpose: Container-ready entry point with standardized paths.

Docker Configuration:

- Input Directory: /app/input - Mount point for source PDFs.
- Output Directory: /app/output - Mount point for generated JSON files.
- Directory Creation: Ensures required directories exist.
- Resource Management: Configures thread pool size for container environment.

Benefits:

- Standardized deployment
- Scalable processing
- Isolated execution environment
- Easy integration with container orchestration

CHALLENGE 1 B: Multi-Collection PDF Analysis